Name : Harsh Phale

Roll No : 2401084

Batch : T21

Experiment No 1

Aim : Perform version control operations using Git

LO 2: To understand and perform the basic version control operations using Git and GitHub, including repository creation, cloning, file tracking, committing, pushing, pulling, branching, merging, and remote repository management.

Part 1: Repository Setup & Configuration

1. Initialize a Local Repository

- **Theory:** Converts a normal folder into a tracked Git repository. It creates a hidden .git directory that serves as the project's database to store all historical versions.
- **Command:** git init

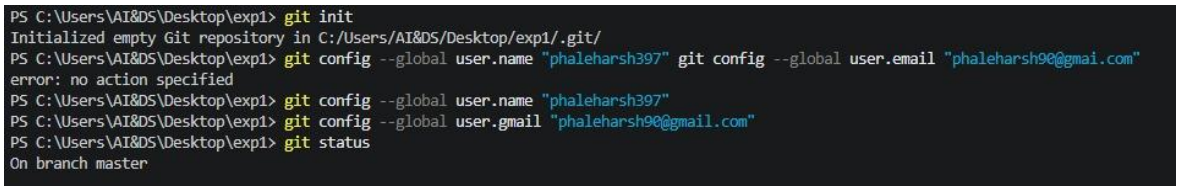


2. Configure Local User Identity

- **Theory:** Sets the author name and email address for your commits. Git requires this so that every historical snapshot is credited to the correct developer.
- **Commands:**

  git config --global user.name "Harsh Phale"

  git config --global user.email phaleharsh90@gmail.com



3. Check Repository Status

- **Theory:** Displays the state of the working directory and the staging area. It shows which files have been modified, which are untracked (red), and which are ready to be committed (green).

- **Command:** git status

```
(base) PS C:\Users\15L\Desktop\ads> git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Part 2: The Core Staging & Committing Workflow

4. Stage a Specific File

- **Theory:** Moves a single targeted file into the Staging Area (Index). This tells Git exactly which file modifications you want to package into your next snapshot.
- **Command:** git add lab.txt

```
(base) PS C:\Users\15L\Desktop\ads> Set-Content lab.txt "Version 1: Core setup."
(base) PS C:\Users\15L\Desktop\ads> git add lab.txt
```

5. Stage All Modified/New Files

- **Theory:** A catch-all shortcut that automatically finds every single new, untracked, or modified file in the folder and moves them into the staging area simultaneously.
- **Command:** git add .

```
(base) PS C:\Users\15L\Desktop\ads> git add .
```

6. Commit Changes

- **Theory:** Permanently saves a historical snapshot of all files currently in the staging area into your local repository database, stamped with a descriptive message.
- **Command:** git commit -m "Initial commit with clean lab file"

```
(base) PS C:\Users\15L\Desktop\ads> git commit -m "Initial commit with clean lab file"
[master (root-commit) d68df0b] Initial commit with clean lab file
 1 file changed, 1 insertion(+)
 create mode 100644 lab.txt
```

7. View Full Commit History

- **Theory:** Displays the complete chronological timeline of all snapshots saved in the project, showing unique SHA-1 tracking hashes, author info, dates, and messages.
- **Command:** git log

```
 1 file changed, 1 insertion(+)
 create mode 100644 lab.txt
PS C:\Users\AI&DS\Desktop\exp1> git log
commit 2ba65b0937e9ca914c3b0cc0e6c5b29bbff58cba (HEAD -> master)
Author: phaleharsh397 <phaleharsh90@gmail.com>
Date:   Thu Jul 23 11:44:08 2026 +0530

    initial commit with clean lab file
PS C:\Users\AI&DS\Desktop\exp1> git log --oneline
2ba65b0 (HEAD -> master) initial commit with clean lab file
```

8. View Condensed History

- **Theory:** Compresses the commit timeline down so each historical snapshot occupies exactly one clean line, making long histories highly scannable.
- **Command:** git log –oneline

```
(base) PS C:\Users\15L\Desktop\ads> git log --oneline
d68df0b (HEAD -> master) Initial commit with clean lab file
```

Part 3: Inspecting & Undoing Modifications

9. Check File Differences

- **Theory:** Computes and displays the exact line-by-line differences between your current uncommitted changes and the last saved snapshot.
- **Command:** git diff

```
(base) PS C:\Users\15L\Desktop\ads> Add-Content lab.txt "`nAdding temporary code here."
(base) PS C:\Users\15L\Desktop\ads> git diff
--- a/lab.txt
+++ b/lab.txt
@@ -1 +1,3 @@
 Version 1: Core setup.
+
+Adding temporary code here.
```

10. Unstage a File

- **Theory:** Safely extracts a file from the staging area if it was added prematurely, reverting its status back to "tracked but unstaged" without losing any of your edits.
- **Command:** git restore --staged lab.txt

```
(base) PS C:\Users\15L\Desktop\ads> git add lab.txt
>> git restore --staged lab.txt
warning: in the working copy of 'lab.txt', LF will be replaced by CRLF the next time Git touches it
```

11. Discard Local Changes

- **Theory:** Erases all uncommitted modifications inside a physical file, instantly rolling its content back to match its state at the last commit.
- **Command:** git restore lab.txt

```
(base) PS C:\Users\15L\Desktop\ads> git restore lab.txt
```

12. Amend the Last Commit

- **Theory:** Modifies your very last commit. It allows you to update the previous commit message or add newly staged files to the last snapshot without creating an entirely new commit block.
- **Command:** git commit --amend -m "Initial commit with simple file updated"

```
(base) PS C:\Users\15L\Desktop\ads> git commit --amend -m "Initial commit with simple file updated"se) PS C:\Users\15L\Desktop\ads>
[master 54fa641] Initial commit with simple file updated
 Date: Wed Jul 15 13:51:07 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 lab.txt
```

Part 4: Branching & Merging

13. List Existing Branches

- **Theory:** Shows all distinct timelines of development in the repo, using an asterisk (*) to mark your active branch.
- **Command:**git branch

```
(base) PS C:\Users\15L\Desktop\ads> git branch
  * master
```

14. Create a New Feature Branch

- **Theory:** Generates an isolated branch pointer splitting off from your current snapshot so you can work safely in isolation.
- **Command:**git branch feature-xyz

```
(base) PS C:\Users\15L\Desktop\ads> git branch feature-xyz
```

15. Switch to a Branch

- **Theory:** Moves your active workspace environment pointer (HEAD) to a different branch timeline.
- **Command:**git switch feature-xyz

```
(base) PS C:\Users\15L\Desktop\ads> git switch feature-xyz
Switched to branch 'feature-xyz'
```

16. Create and Switch Simultaneously

- **Theory:** A handy combined command that cuts down steps by creating a new branch and instantly stepping into it.
- **Command:**git switch -c rapid-feature

```
(base) PS C:\Users\15L\Desktop\ads> git switch -c rapid-feature
Switched to a new branch 'rapid-feature'
```

17. Merge a Branch

- **Theory:** Brings the changes from an isolated timeline back into your primary production pipeline.
- **Commands:** git switch master

git merge feature-xyz

```
(base) PS C:\Users\15L\Desktop\ads> git switch master
>> git merge feature-xyz
Switched to branch 'master'
Already up to date.
```

18. Delete a Branch

- **Theory:** Destroys an isolated branch pointer once its changes have been safely merged away.
- **Command:** git branch -d feature-xyz

```
(base) PS C:\Users\15L\Desktop\ads> git branch -d feature-xyz

    -f, --[no-]fetch      fetch the remote branches
    --[no-]tags           import all tags and associated objects when fetching
                          or do not fetch any tag at all (--no-tags)
    -t, --[no-]track <branch>
                          branch(es) to track
    -m, --[no-]master <branch>
                          master branch
    --[no-]mirror[=(push|fetch)]
                          set up remote as a mirror to push to or fetch from
```

19. View Linked Remotes

- **Theory:** Verifies your external connections by displaying the exact active cloud URLs mapped for pulling (fetching) and pushing (uploading) data.
- **Command:** git remote -v

```
(base) PS C:\Users\15L\Desktop\ads> git remote remove origin
```

20. Push Changes Online

- **Theory:** Uploads your local repository commits, branches, and histories directly into your hosted repository cloud space on GitHub.
- **Command:** git push -u origin master

```
(base) PS C:\Users\15L\Desktop\ads> git push -u origin master
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 260 bytes | 260.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Yomesh-P/git-lab-experiment.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

21. Fetch Remote Changes

- **Theory:** Connects to the remote server and downloads new metadata and history records added by collaborators, without altering your active files.
- **Command:** git fetch origin

```
(base) PS C:\Users\15L\Desktop\ads> git fetch origin
```

22. Pull Remote Changes

- **Theory:** Automatically runs a two-part combination: it downloads updates from the online repository (fetch) and instantly integrates them into your current local branch files (merge).
- **Command:** git pull origin master

```
(base) PS C:\Users\15L\Desktop\ads> git pull origin master
From https://github.com/Yomesh-P/git-lab-experiment
 * branch            master     -> FETCH_HEAD
Already up to date.
```

Part 6: Temporary Workspace Storage

23. Stash Temporary Work

- **Theory:** Takes your modified, uncommitted changes and places them on a temporary storage shelf, giving you a completely clean workspace without forcing you to make an incomplete commit. Running git stash pop pulls them back off the shelf later.
- **Command:** git stash

```
(base) PS C:\Users\15L\Desktop\ads> git stash
Saved working directory and index state WIP on master: 54fa641 Initial commit with simple file updated
(base) PS C:\Users\15L\Desktop\ads> git stash pop
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   lab.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        git-lab-experiment/

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (2be8ac70f56b6fe2785faa1669a429c2c87ac571)
```

Part 7: Authentication & Rollback Control

Authorization Configuration

- **Theory:** This command updates Git's authentication settings to use the operating system's native credential manager. It triggers a secure visual pop-up window or browser tab to handle your GitHub login tokens without freezing your terminal interface.
- **Command:**git config --global credential.helper manager

```
(base) PS C:\Users\15L\Desktop\ads> git config --global credential.
helper manager
```

Rollback Operations (Undoing Commits)

When working with version control, you sometimes need to undo a committed snapshot. Git offers two primary rollback strategies depending on whether you want to preserve or erase history:

1. Non-Destructive Rollback (Create a Reversal Commit)

- **Theory:** This command rolls back changes safely by calculating the exact opposite of a specific past commit and automatically generating a brand-new commit containing those reversed changes. This is the best practice for collaborative environments because it preserves your entire history tree instead of erasing past events.
- **Command:**git revert <commit-hash>

```
(base) PS C:\Users\15L\Desktop\ads> git log --oneline
b650c25 (HEAD -> master, origin/master, origin/HEAD) Completed all
25 Git operations for the lab experiment
54fa641 (rapid-feature) Initial commit with simple file updated
(base) PS C:\Users\15L\Desktop\ads> git revert 54fa641 --no-edit
CONFLICT (modify/delete): lab.txt deleted in (empty tree) and modif
ied in HEAD.  Version HEAD of lab.txt left in tree.
error: could not revert 54fa641... Initial commit with simple file
updated
hint: After resolving the conflicts, mark them with
hint: "git add/rm <pathspec>", then run
hint: "git revert --continue".
hint: You can instead skip this commit with "git revert --skip".
hint: To abort and get back to the state before "git revert",
hint: run "git revert --abort".
hint: Disable this message with "git config set advice.mergeConflic
t false"
```

2. Destructive Rollback (Reset the Timeline Back)

- **Theory:** This command physically rolls your active timeline pointer back to a specific target commit hash. Using the --hard flag completely discards all code modifications and commits made *after* that target snapshot, erasing them from your history tree permanently.
- **Command:**git reset --hard <commit-hash>

```
(base) PS C:\Users\15L\Desktop\ads> git revert --abort
(base) PS C:\Users\15L\Desktop\ads> git reset --hard 54fa641
warning: unable to rmdir 'git-lab-experiment': Directory not empty
HEAD is now at 54fa641 Initial commit with simple file updated
(base) PS C:\Users\15L\Desktop\ads> git log --oneline
54fa641 (HEAD -> master, rapid-feature) Initial commit with simple
file updated
```

The Final Execution: Project Close-Out

To officially seal your experiment and upload all your hard work to the cloud, you executed these final operations:

1. git add .

- **Theory:** Scans your entire directory for the final changes (including the file changes recovered from your stash operation) and moves them all safely into the staging area so they are queued up for the final snapshot.

2. git commit -m "Completed all 25 Git operations for the lab experiment"

- **Theory:** Captures the absolute final state of your experiment workspace, archiving your success permanently in your local Git history log with a clear conclusion message.

3. git push origin master

- **Theory:** Transmits your entire updated history timeline straight up to GitHub, syncing your final local changes to the cloud so your submission is safely backed up and visible on your repository page.

```
(base) PS C:\Users\15L\Desktop\ads> git add .
>> git commit -m "Completed all 25 Git operations for the lab experiment"
>> git push origin master
warning: adding embedded git repository: git-lab-experiment
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents of
hint: the embedded repository and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint:   git submodule add <url> git-lab-experiment
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:
hint:
hint:   git rm --cached git-lab-experiment
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"
[master b650c25] Completed all 25 Git operations for the lab experiment
 2 files changed, 3 insertions(+)
 create mode 160000 git-lab-experiment
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 387 bytes | 387.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Yomesh-P/git-lab-experiment.git
   54fa641..b650c25  master -> master
```

Conclusion

By hands-on execution of this laboratory experiment, I learned how to manage a complete software lifecycle using Git version control tools. Specifically, I gained practical knowledge in:

- **Workspace Control:** I learned how to initialize databases, configure user profiles, and track changes across the working directory, staging area, and commit history.
- **Error Correction and Safety:** I mastered techniques to unstage files, discard local mistakes, stash temporary work-in-progress code without committing it, and resolve operational conflicts.
- **Timeline Travel:** I learned how to create isolated feature branches to safely experiment with code modifications and merge them back into the production pipeline.
- **Cloud Integration:** I learned how to establish secure authentication links to GitHub, pushing local commits online and fetching remote updates to maintain environment synchronization.
- **History Recovery:** I learned the functional difference between non-destructive history preservation (git revert) and destructive timeline rewinding (git reset --hard).

Ultimately, doing this experiment taught me how to maintain full codebase integrity, track historical engineering changes, and prevent data loss, which are foundational skills for collaborative software engineering and data science workflows.